May, 2023

Modular Serial-Chain Robotic Arm with Plug-and-Play Architecture for Effective Teaching and Learning

A PROJECT REPORT

Submitted by

BL.EN.U4MEE19002	Ashwin S Kumar
BL.EN.U4MEE19009	Gaurav Shankar
BL.EN.U4MEE19023	Manjunath Sakthivel

in partial fulfillment for the award of the degree 0f

BACHELOR OF TECHNOLOGY

IN

MECHANICAL ENGINEERING

AMRITA VISHWA VIDYAPEETHAM

DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF THE UGC ACT, 1956

AMRITA SCHOOL OF ENGINEERING, BANGALORE

AMRITA VISHWA VIDYAPEETHAM

BANGALORE 560 035

May-2023

AMRITA VISHWA VIDYAPEETHAM
AMRITA SCHOOL OF ENGINEERING, BANGALORE, 560035

May, 2023

AMRITA VISHWA VIDYAPEETHAM

DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF THE UGC ACT, 1956

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "**MODULAR SERIAL-CHAIN ROBOTIC ARM WITH PLUG-AND-PLAY ARCHITECTUE FOR EFFECTIVE TEACHING AND LEARNING"** submitted by

Ashwin S Kumar	BL.EN.U4MEE19002
Gaurav Shankar	BL.EN.U4MEE19009
Manjunath Sakthivel	BL.EN.U4MEE19023

in partial fulfillment of the requirements for the award of the **Degree Bachelor of Technology** in **MECHAINICAL ENGINEERING** is a bonafide record of the work carried out under my(our) guidance and supervision at Amrita School of Engineering, Bangalore. and "…name of the organization in case of external work".

Dr. Rajeevlochana G. Chittawadigi
Asst. Professor (Sr.Gr.)
Department of Mechanical Engineering.

NAME OF CHAIRPERSON- Dr. Phanibhushan M.V.

This project report was evaluated by us on …… (Date...)

INTERNAL EXAMINER EXTERNAL EXAMINER

ACKNOWLEDGEMENT

First of all, we are grateful to **Dr. Phanibhushan M.V.** Vice Chairperson, Department of Mechanical Engineering for giving us this opportunity to showcase our skills in multiple disciplines and concepts.

We are fortunate to have the mentorship of our project guide **Dr. Rajeevlochana G. Chittawadigi**, Assistant Professor (Sr.Gr.), Department of Mechanical Engineering, Amrita School of Engineering, Bangalore, who was willing to provide us with appropriate supervision at the time of need. We were able to reach this stage with the support provided by him. His exemplary guidance, careful monitoring and constant encouragement has helped us power through various hurdles and achieve our goal.

We would also like to use this opportunity to show our gratitude to our evaluators **Dr. Ravi Kumar**, Department of Mechanical Engineering, Amrita School of Engineering, Bangalore, and **Dr. Prashanth B.N**, Department of Mechanical Engineering, Amrita School of Engineering, Bangalore, who have helped us with their insightful thoughts, advices, and suggestions. We are thankful for the support provided to us by the lab staffs Mr. Money and Mr. Dharmendra, who were willing to help us with their experience and skill while operating the different machinery in the labs and workshop. We would also like to show our gratitude to Mr. Hrishi from Sameta Metal Pro Pvt. Ltd., who has helped us in manufacturing the sheet metal components.

Last but certainly not the least, we would like to thank all the teaching and non-teaching task of the Department of Mechanical Engineering, Amrita School of Engineering, Bangalore and all family, friends, and other benefactors for providing us with support and strength at the time of need due to which we were able to accomplish our goals with relative ease.

Thank You.

ABSTRACT

In the past few decades' robotics have become a topic of huge importance. Automation has become the new talk of the town. To keep up with changing time, Universities have started to include various Robotics courses in their curriculum. Industrial Robotics is one such course where topics on various serial chained Industrial Robotics, the sensors used, the kinematics and dynamics used, automation od parts etc., are taught in detail. Serial chain robotic arms are manipulators that are connected in series to one another with links and joints. These joints provide the manipulator with motion, each joint usually has one DOF (Degree of Freedom). The motion of the manipulator is determined with the help of the joint axes of each joint.

One important concept is DH (Denavit-Hartenberg) parameters, which is essential for understanding robot kinematics of serial robotic arms. This concept is used to represent the transformation of any joint axes for one to another. This topic is relatively hard to understand, hence we have come up with an ideology to help facilitate the learning of this concept.

Modularity is brought into picture, with which the concept of DH parameters can be learned with relative ease, as a robotic manipulator can be assembled with modular components discussed below. With this, the students will have a hands-on experience in assembling and disassembling a manipulator arm. This will help them make a robotic arm of their choice and configuration.

TABLE OF CONTENTS

1. **Introduction and Objective.**
2. **Literature Review.**
   - 2.1 Offline Simulation of Motion Planning of a Planar Manipulator in RoboAnalyzer and its Integration with a Physical Prototype.
   - 2.2 Kinematic Analysis and Development of Simulation Software for Nex Dexter Robotic Arm.
   - 2.3 Configuration Independent Kinematics for Modular Robots.
   - 2.4 Optimal Wrist Design of Wrist-Hollow type 6-axis Articulated Robot using Genetic Algorithm.
   - 2.5 Modelling and Simulation of Basic Robotic Configurations using DH parameters based Adaptive Models.
3. **Serial Chain Robotic Arms**
   - 3.1 Types of Robotics Arms
   - 3.2 Use of Robotic Arms in Education Sector.
   - 3.3 Forward Kinematics.
   - 3.4 Inverse Kinematics
   - 3.5 Homogeneous Transformation Matrix.
   - 3.6 DH (Denavit-Hartenberg) Parameters
4. **Modular Components and RAMG (Robotic Arm Model Generator).**
   - 4.1 3D CAD Model
   - 4.2 Physical Prototype
5. **Motion Planning and Programming.**
6. **Achievements**
   - 6.1 Modular Serial-Chain Robot with Plug-and-Play Architecture for Effective Teaching and Learning. (Research Paper)
   - 6.2 Virtual and Physical Components to Build Modular Robotic Arms for Effective Understanding of Denavit-Hartenberg (DH) Parameters. (Research Paper)

6.3 Provisional Patent Applied with title Modular Serial-Chain Robot with application number 202341022970

7. Conclusion

Reference

Appendix

INDEX

List of Figures

Figure 3.1: - Image of a 2R robotic arm which us used for the derivation of forward kinematics.
Figure 4.1: - Dynamixel AX-12 A motor made by Robotis which is used as an actuator.
Figure 4.2: - Motor clamp, which is used to attach the motor to other components.
Figure 4.3: - Adapter, consists of holes which we can screw through to joint links and joints.
Figure 4.4: - Offset adapter, this has the screw holes in different positions to facilitate easier attachment of components.
Figure 4.5: - Straight link, used to connect the joints with the help of the adapter.
Figure 4.6: - Wrist link, used only the Spherical Wrist robot for the creation of the wrist point.
Figure 4.7: - Universal Base, helps in mounting the entire robotic arm, and acts as a counter weight.
Figure 4.8: - Electromagnetic gripper, used to pick and place objects with help of electromagnetic concept.
Figure 4.9: - Parallel sub-assembly, which is one of the building blocks of the model.
Figure 4.10: - Intersecting sub-assembly, the joint axes intersect at a point in this model.
Figure 4.11: - Skew sub-assembly, the joint axes in this sub-assembly are not parallel not intersecting.
Figure 4.12: - Flow chart for RAMG (Robot Arm Model Generator), shows its process flow.
Figure 4.13: - 2R planar virtual model made with the modular components.
Figure 4.14: - 3R planer virtual model
Figure 4.15: - 3R Spatial virtual model.

Figure 4.16: - 6R Spherical Wrist configuration's virtual model.
Figure 4.17: - 6R Offset Wrist configuration's virtual model.
Figure 4.18: - Virtual model of Dual Arm configuration which is special type of architecture.
Figure 4.19: - Physical model of 3R spatial manipulator.
Figure 4.20: - Physical model of 2R Planar manipulator.
Figure 4.21: - Physical model of 3R Planar manipulator.
Figure 4.22: - Physical model of 6R Spherical Wrist manipulator.
Figure 4.23: - Physical model of 6R Offset Wrist manipulator.
Figure 5.1: - Flowchart which represent the flow process of which helps in controlling the motors.
Figure 5.2: Screenshot from Windows forms application which shows the forward kinematics programs inputs,
Figure 5.3: - Screenshot from Windows forms application which show the joint jogging and CSV file program.

List of Tables

Table 3.1: - DH parameters of 2R Planar manipulator.

Table 3.2: - DH parameters of 3R Planar manipulator.

Table 3.3: - DH parameters of 3R Spatial manipulator.

Table 3.4: - DH parameters of 6R Spherical Wrist manipulator.

Table 3.5: - DH parameters of 6R Wrist Offset manipulator.

1. Introduction and Objective

In multiple Universities robotics courses are taught in their curriculum as a part of their undergraduate and postgraduate programs. They involve various courses related to the architecture, configuration, and different features of serial chain robots or manipulators. In such manipulators, the links are connected to one another in series with the help of various joints. These joints help in the movement of the robotic arm. Each joint provides the manipulator with one DOF (Degree of Freedom).

Industrial Robotics is one of the commonly taught robotics courses as it deals with the concepts related to various robots used in the industry. After the completion of this course, students will have extensive knowledge in the types of robots, various sensors used, dynamics and kinematics of robotic arm, DH (Denavit-Hartenberg) parameters [1] etc., To understand the kinematics of a robotic arm DH parameters is essential. As this concept is relatively difficult to understand, we have come with a product to help facilitate easier learning of DH parameters.

This product follows the modularity path, where various modular components are used to create a robotic manipulator of the desired configuration and architecture. Because of which, robots of multiple architecture can be made according to the user needs. This can help students get a hands-on experience of actual physical models of a robotic arm, and make a working model bases on their creativity and imagination. This can also help students understand the concept of DH parameters easily. Students can perform interactive activities like assembling a robotic arm when its DH parameters are provided, also calculating the DH parameters when the physical model is given. This will improve the creative thinking of the students along with the enhancing of their knowledge in various robotics concepts.

This same product can be used for various other applications like teach pendant for

programming various other large robots. Because of the products modular architecture, a robot of any configuration can be controlled.

The objective of this project is discussed below.
In the Industry, robots of different architectures are used for multiple purposes. As various configurations are available, it is inefficient to have all these configurations. Hence a modular robotic kit is to be developed through which students can understand the various kinematics and dynamics of a robotic arm.
Here the users can change the configuration of the robotic arm with the help of simple tools in very less time.
The main objective is for students to easily understand the concept of DH (Denavit-Hartenberg) parameters, and to get hands on experience of actual physical working prototypes.
This will make the learning experience of the students much more intuitive and will help them foster a creative and imaginative mindset. This can help students with multiple real-world problems, and help them come up with creative and unique solutions.

2. Literature Review

This section deals with the various reviews done on the existing and similar technologies and methods related to the objective of the project. This has given us a better insight on the different nomenclatures and concepts. Some of these papers are explained in brief below.

2.1 Offline Simulation of Motion Planning of a Planar Manipulator in RoboAnalyzer and its Integration with a Physical Prototype [2]

In this paper the author has worked on an offline simulator planning the motion of a 2R planar manipulator in RoboAnalyzer [3] and Integrating It with the physical prototype. There are two ways to look at robotics education, learning from the mathematics of the robot and development of the physical prototype of the type of robot by having hands on experiences. As the author has mentioned there are a lot of resources available to teach and learn the mathematics of the robotics but there are lesser resources for the students to learn the building structures and the physical prototypes for the better understanding of the robotics education.

This paper proposes a new motion planning for module in roboanalyzer (A Robotic Simulator Software) in a 3d structure. It has a very simpler interface developed by the author to have an interactive experience with a physical prototype using a simulating software.

Here the author inputs the parameters of a 2R planar Manipulator and perform he kinematics analysis and motion planning to get the circular and the linear motion through the end effector end point. Once it is ready the joint trajectories can be sent through the Arduino IDE through the COM serial Port in communication with the laptop placed to receive the trajectories from the

physical prototype using Arduino IDE. Author also explains how to demonstrate the prototype developed and how it's useful for the learning and understanding of the robotics and how simpler it is to use for higher understanding and hands on experience with the 3D model.

2.2 Kinematic Analysis and Development pf Simulation Software for New Dexter Robotic Arm [4]

Here the author uses a pre-existing robotic arm called Nex Dexter 5 axis manipulator. Here they try to develop a robot simulator application using C# to connect the robot module with another application called RoboAnalyzer and visualize the movement of the robot before its actual movement. The application name is Teach Pendent, the robotic kinematics analysis was formulated for the join and cartesian motion.

Here the author has developed the 3d model of the Nex-Dexter robot [5] from the dimensions and used inventor pro to create the model of the robot. The DH parameters of the robot is being extracted from the technical drawing of the robot arm and the step to be followed also are mentioned.

Using the DH parameters, the robot frame is being run with the values of the robot architecture, here there is mention of the motion in forward kinematics and inverse kinematics of the robot and its behavior.

The C# application runs by taking the user input and by jogging where the values are incremented or decreased according to the angle mentioned. Virtual robots make motion, and the same is being done by the offline robot.

2.3 Configurations of Independent Kinematics for modular Robots [6]

Here the authors are developing a modular robotic system which has standardized links sizes and joints which can me later modified into different kinematic configurations and the modular components are designed in cubical structures giving them a modular look and easy to connect to each face. They used a scheme called AIM (Assemble Incident Matrices) to change the configurations of the robotic arm.

Here the authors have used dyad kinematics to determine the location of the robotic arm. The tree configuration has been taken to explain the iterative forward kinematics of the robotic arm where the algorithm is used to derive the forward transformation from the base link to all the links automatically using AIM.

Two series of link modules and 2 of joint module and connector or adaptor are being used in this paper. Other types of adaptors can be designed when needed of the configuration.

2.4 Optimal Wrist Design of Wrist-Hollow type 6-axis Articulated Robot using Genetic Algorithm [7]

In this paper the author has an optimized design of wrist point hollow type 6-axis robot for making a wrist hollow arc welding robotic arm which can shorten the welding time and the quality of arc welding.

The optimized design of hollow arms and width of upper arm by using the simulation of robot motion.

The authors have used software's like solid works for robot modelling and MATLAB to GS optimization and RecurDyn to show the characteristics of the robot on its movements.

The authors have developed a welding specialized robot, which is 6 axes. The axes 1,2,3 and 5 are revolute joints and axes 4 and 6 are hollow type of joints

which contain the welding cables and the motors. They used GA simulation to generate a new a more flexible type of welding robot which has more reach and more practicality.

2.5 Modelling and Simulation of Basic Robotic Configurations using DH parameters based Adaptive Models [8]

In this paper the author has discussed three types of modules which can be used in the robotic manipulator namely basic length unit adaptive twist length and expandable unit length. Here the design of the twist angle is adjusted so both types of twist angle are achieved.

As mentioned, the twist angle is either 90◦ or 0◦ but in the twist angle adjustment component the flexibility to develop the manipulator which has a twist angle of 90◦ or 0◦ and has two different planes.

3. Serial Chain Robotic Arms

A robotic arm is manipulator with multiple links and joints. A link is connected to another link with the help of joint. A joint usual gives a manipulator one DOF (Degree of Freedom) of motion. The higher the number of DOF's the more flexible the manipulator will be. The amount of range and motion also relies on the number of DOF's. These arms are used for different applications in multiple industries. This can range from simple pick and place applications to complex medical surgery procedure. These robots run on a series of programs which is sent to the robot via a controller. They also consist of multiple sensors for different parameters like the joint angles, torque, velocity, etc., depending on this the controller can take necessary action and control to robot. The different types of robots and the use of such manipulators is explained below. A serial chain robotic arm is has its links and joint attached to one another in series, which gives it the shape of an arm.

3.1 Types of Robotic Arms

There are multiple varieties of robotic arm available, which are used for different applications. These robotic arms are distinguished bases on their DOF's, their configurations, architecture, coordinate system etc., these are explained in detail below.

Before understanding the types of robotic arms, it is essential to know the type of joint that are there in a robotic arm. There are primarily two types of joints in a robotic arm, there are prismatic and revolute joints. Prismatic joints provide the robotic arm with linear motion whereas revolute joints provide the manipulator with rotation joints. Depending on the application the appropriate joint is selected.

Based on the type of coordinate system:

i) Cartesian coordinate system: This type of robots has three prismatic joints. The work envelope of this robot is in the shape of a cube or cuboid.

ii) Cylindrical coordinate system: In this robot, there are two prismatic

joints and one revolute joint. The work envelope is in the shape of a cylinder, hence the name cylindrical coordinate system.

iii) Spherical coordinate system: Here the robot has two revolute joints and one prismatic joint. This robot has a work envelope of a sphere.

iv) Jointed arm coordinate system: In this case the robot has three revolute joints. This is similar to a human arm when it comes to its motion. This provides the robotic arm with the highest range of motion with the least amount of floor space being occupied.

The robots can also be differentiated based on some parameters, they are

a) Payload: The amount of load that a robot can carry successfully.
b) Velocity: The maximum speed at which the end effector of a robotic arm can move.
c) Cycle: Time taken by the robotic arm to complete one cycle of motion.
d) Accuracy: Ability of a manipulator to get the end effector to the desired position.
e) Repeatability: Ability for the manipulator to come back to its pre-defined position.
f) Resolution: The smallest movement possible by the robotic arm.
g) Size: The actual size of the robotic arm.

Based on the architecture, the robotic arms can be differentiated. Some of them are shown below.

1. 2R Planar Manipulator: This configuration has two joint axes which are parallel to one another. The work envelope is on a single plane.
2. 3R Planar Manipulator: Here the manipulator is similar to the 2R manipulator but has three joint axes instead of the two.
3. 3R Spatial Manipulator: This manipulator has three joint axes and has a work envelope in 3-dimensional space.
4. 6R Spherical Wrist Manipulator: There are six joint axes in this type of manipulator, hence giving it six DOF's. In this configuration, the last three joint axes intersect at a point, this point is known as a wrist point. This

satisfies Pieper's first criterial for a robotic arm. This is one of the commonly used types of architecture for a serial chain robotic arm in the industry.

5. 6R Offset Wrist Manipulator: This configuration is similar to the Spherical Wrist configuration. But instead of having three joint axes that intersect at a point, they have three axes that are parallel to each other, in this case they are the second, third and fourth joint axes. These are another type of configuration which is commonly used in the industry.
6. Dual arm configuration: This is a special type of architecture in which the robotic manipulator has two arms, each capable of doing its own motion. This configuration has 15 DOF's and can perform any operation which is performed by two human arms.

The images and CAD model of these configurations are shown in the section below.

3.2 Use of Robotic Arms in Education Sector

To teach robotics concept, universities use multiple online and offline/physical tools to make the learning experience more interactive. Physical models of robotic arms are used in universities to facilitate the learning of robotics concepts. Multiple online tools are also used to enhance the teaching and learning experience for the students. Some of such online tools are software like RoboAnalyzer, RoboDK etc., The physical models let students have a hands-on experience of the actual working and functioning of a robotic arm, and the various principle, concepts and components used for the successful operation of the manipulator. They are given the opportunity to control a robotic arm with the help of the relevant software installed in a computer which is connected to the robotic arm. Some examples of this physical prototype are KUKA robots [9], PUMA robots [10] etc. Each of these robots have a distinct architecture which cannot be changed.

Students are given the opportunity to program and control these robots which lets them have a much more deeper understanding of the functioning of the robotic arm.

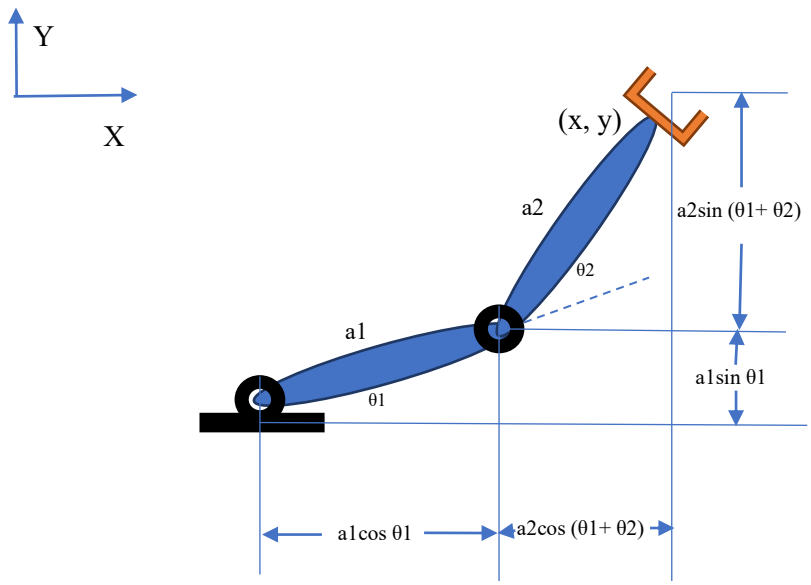
Hence these tools help the students have a better and in-depth understanding of the robotic concepts and the working of a physical model. They also learn how to control and program a robotic arm, which will help the students with a lot of real-world application problems.

To understand the proper functioning of a robotic arm, it its necessary to be knowledgeable in various concepts related to robotics. There are few concepts that hold the highest level of importance, as it is impossible to control a robot without the help of these concepts. They can be called the basic building blocks of robotics. Any student who completes the course of Industrial Robotics must have a comprehensive knowledge of these concepts. These concepts are explained in detail below. There are other concepts which are not discussed. These are related to the types of joints used by the manipulator, the type of sensors used, the different types of end-effectors used for different applications. The type of power source used by the robotic arm are electric, hydraulic, and pneumatic. Depending on the application of the robots, the power source can be selected.
The different coordinate frames are the World Coordinate Frame, Base Coordinate Frame, Joint Coordinate Frame, Tool Coordinate Frame etc. These frames are used to represent the position of the different joints, robot, end-effector etc., relative to another point. The basic concepts that are fundamental requirement for a robots functioning is discussed in detail below.

3.3 Forward Kinematics

Forward kinematics is a robotics concept which is used to control the motion or to program a robot. But before understanding forward kinematics, it is essential to understand what kinematic is. Kinematics is branch of mechanics which deals with the study of motion related to objects without considering the cause or force which initiated that motion. This is necessary to program the robotic arm for different applications.
Forward kinematic is the robotic concept related to the robotic arm in which the

joint parameters, that is the joint values are known but not the final position of the end-effector. Forward kinematics equations are easier to derive and understand and are used in the industry when the function which is to be performed by the robot is simple. Forward kinematics for a simple Planar 2R robotic arm is shown below with the appropriate derivations and equations.



$$x = a1 \cos \theta1 + a2 \cos (\theta1 + \theta2) \ldots\ldots\ldots (1)$$
$$y = a1 \sin \theta1 + a2 \sin (\theta1 + \theta2) \ldots\ldots\ldots (2)$$

Fig 3.1: Forward Kinematics

3.4 Inverse Kinematics

This is another robotics concept which is essential for understanding the programming and controlling of a robotic arm. In inverse kinematics, the joint variables are not known by the user, but the final position of the end-effector is known. Because of this there are more than one solution possible for the end-effector to reach its final position.

This is one of the most widely used concept to program a robotic arm as it is easier and complex motions can easily be achieved with the help of inverse kinematics. Depending on the position and orientation of the manipulator, the controller can

choose the appropriate path from the multiple solution which it has calculated. This saves a lot of time and energy and is the most efficient method to do so. This concept is yet to be explored in detail based on the applications, and depending upon the user's requirements. Since the product is modular in nature the inverse kinematics of the model becomes much more competent compared to any other robotic arm with a fixed configuration.

3.5 Homogeneous Transformation Matrix

To represent the movement of one a frame or joint from one position to another, a transformation matrix is needed. This is 4X4 matrix in which the first 3X3 part of the matrix represents the rotation of the joint, the last column represents the translation along the all the three axes (X, Y, and Z), and the last row denotes the scale factor of the matrix.

Before discussing about the matrix in detail, it is necessary to understand the types of frames, and different matrix used for specific situations. There are two types of frames on is fixed frame which is denoted by x, y and z. The position of this frame is always fixed. The second type is the moving frame, which is denoted by u, v and w. The position of this frame keeps changing depending on the values.

There are also rotational matrix and translation matrix. Rotation matrix only signifies rotational motion whereas translative matrix signifies only translation motion. When both these matrices are combined, we get a combined translation matrix. This matrix is extremely helpful when it comes to programming and controlling the motion of the robotic arm.

The homogeneous transformation matrix for rotation along X axis, Y axis and Z axis, as well a translation matrix along with their combined matrix.

When the multiple matrices of a robotic arm are multiplied in a certain order, we get the final transformation matrix for the robotic arm, this gives the position of the end-effector with respect to the base of the manipulator. These matrices can be used for both forward and inverse kinematics approach.

If we have the values of the matrix in its forward kinematics form, then the inverse of it can easily found by inverting the matrix.

HTM (Homogeneous transformation matrix)

$$\begin{bmatrix} n_x & o_x & a_x & P_x & n_y & o_y & a_y & P_y & n_z & o_z & a_z & P_z & 0 & 0 & 0 & 1 \end{bmatrix}$$
……(3)

ROT(X, θ) Axis

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & cos\,cos\,\theta & -sin\,sin\,\theta & 0 & 0 & sin\,sin\,\theta & cos\,cos\,\theta & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$
……(4)

ROT(Y, θ)

$$\begin{bmatrix} cos\,cos\,\theta & 0 & sin\,sin\,\theta & 0 & 0 & 1 & 0 & 0 & -sin\,sin\,\theta & 0 & cos\,cos\,\theta & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$
……(5)

ROT(Z, θ)

$$\begin{bmatrix} cos\,cos\,\theta & \theta & 0 & 0 & sin\,sin\,\theta & cos\,cos\,\theta & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$
……(6)

3.6 DH (Denavit-Hartenberg) Parameters

DH (Denavit-Hartenberg) parameters is one of the most important concepts when it come to a serial chain robotic arm. This is essential to find the relative position of one joint relative to another. Here the joint axes are represented as lines and the position of these lines are found with respect to one another.

Usually, to represent any line in a three-dimensional space, six variables are needed, to represent the two points of a line along all the three axes (X, Y and Z). This makes the calculation much lengthier and more inefficient, hence to address this

issue, Denavit and Hartenberg came with a concept known as DH parameters.
When the concept of DH parameter is implemented, instead of using six variables to define a line, the same line can be defined with the help of only four parameters. These four variables together are known as DH parameters, and the joint axes which these parameters represent are known as DH frame.
Out of the four parameters, two are angular parameters, and the other two are translative parameters.
The four DH parameters are, θ joint angle, which denotes the number of degrees by which the joint axis, i.e., Z axis rotates, this is represented in degrees. The next parameter is b joint offset, which denotes the distance translated along Z axis, it is represents in mm. α represents twist angle. This show the amount of rotation made by the frame along the X axis and is represented in degrees. The last DH parameter is a which is known as link length. This shows the total distance translated along X axis by the DH frame.

From these four variables, we can find the position of one DH frame with respect to another, and this makes the calculation and process much simpler.
For the derivation of the DH parameters, we take two lines that are skew to each other. Our goal is to translate from one of these line to another. These two lines represent the Z axis of the frame. The X axis of the frame is also considered for the derivation purpose. The Y axis is neglected. Since the line the lines are skew, another line is drawn to connect these lines which are normal to both the Z axes. This connecting line is known as common normal.

Starting off, we rotate the frame about Z axis, such that the X axis of that frame is in the same direction of the common normal, this value is known as joint angle and is represented with the symbol θ.
After this, the frame is translated along its Z axis till the X axis of the frame coincides with the common normal. The number of units translated by the frame is known as joint offset and is represented by the symbol b.
After completing this, the frame is then translated along its X axis till the origin of the frame coincides with the point where the common normal meets the second

axis. This is known as link length, and is represented by *a*.

Finally, the frame is rotated about its X axis such that its Z axis aligns with the second line (Z axis of second line). This parameter is known as twist angle and is represented by *α*.

After getting these values for all the joint axes, a DH table can be formulated and all the necessary information can be inferred from it. The values of this table are post multiplied as they are calculated with respect to the moving frame.

The DH parameters tables for some of the basic configurations of 2R Planar, 3R Planar, 3R Spatial, 6R Spherical Wrist. 6R Wrist Offset, and Dual Arm is shown below. These tables help us understand the architecture of the robotic arm along with its dimensions. It is also used for the kinematics of the manipulator and for controlling it.

Joint No.	Joint Offset(*b*) (mm)	Joint Angle (θ) (°)	Link Length (*a*) (mm)	Twist Angle (*α*) (°)
2R Planar				
1	0	θ	138.323	0
2	0	θ	26.49	0

Table 3.1: DH parameters for 2R Planar manipulator.

Joint No.	Joint Offset(*b*) (mm)	Joint Angle (θ) (°)	Link Length (*a*) (mm)	Twist Angle (*α*) (°)
3R Planar				
1	0	θ	138.323	0
2	0	θ	108.5	0
3	0	θ	26.49	0

Table 3.2: DH parameters for 3R Planar manipulator.

Joint No.	Joint Offset(*b*) (mm)	Joint Angle (θ) (°)	Link Length (*a*) (mm)	Twist Angle (*α*) (°)
3R Spatial				

1	142.2	(	0	90
2	0	(	108.5	0
3	49.34	(	0	0

Table 3.3: DH parameters for 3R Spatial manipulator.

Joint No.	Joint Offset(b) (mm)	Joint Angle( ) (°)	Link Length(a) (mm)	Twist Angle(α) (°)
6R Spherical Wrist				
1	142.2	(	0	90
2	0	(	108.5	0
3	21.96	(	0	0
4	168.9	(	0	90
5	0	(	0	270
6	101.4	(	0	0

Table 3.4: DH parameters for 6R Spherical Wrist Manipulator.

Joint No.	Joint Offset(b) (mm)	Joint Angle( ) (°)	Link Length(a) (mm)	Twist Angle(α) (°)
6R Wrist Offset				
1	142.2	(	0	90
2	0	(	108.5	0
3	0	(	108.5	0
4	71.8	(	0	90
5	0	(	108.5	270
6	49.34	(	0	0

Table 3.5: DH parameters for 6R Wrist Offset Manipulator.

Joint No.	Joint Offset(b) (mm)	Joint Angle( ) (°)	Link Length(a) (mm)	Twist Angle(α) (°)
Dual Arm				
1	142.7	(	0	90
2	116.8	(	0	90
3	0	(	0	270

4	286.89	(	0	90
5	0	(	0	270
6	286.89	(	0	90
7	0	(	0	270
8	0	(	53.3	0

Table 3.6 : DH parameters for one arm of the Dual Arm manipulator (both the arms have the same DH parameter).

4. Modular Components

Different components are used to assemble the robotic arm to the user's desired configuration. These consists of links (fig 4.6 and 4.7), motors (fig 4.2), motor clamps (fig 4.3), adaptor (fig 4.3 and 4.5) and universal base (fig 4.8). The motors act as a joint for the robotic arm. When these components are arranges in an appropriate order, different configurations can be achieved like 2R planar, 3R planar, six axis spherical wrist, etc., the motor used in this is the Dynamixel AX-12A [11] motors from Robotis, which has an accuracy of 0.29 degrees. To control these motors a controller from Robotic is used which is known as U2D2 [12]. The details of these components are discussed in detail in the Motion Planning section. A computer is used with a program to run the motors and move the manipulator to the desired position of the user.

These modular components are assembled in a particular order to crate sub-assemblies. These sub-assembles when combined generate the robotic manipulator of the desired architecture. These sub-assembles are classified based on their relation between two joint axes. They are parallel (fig 4.9), when the axis of rotation of both the joints are parallel to each other. Intersecting (fig 4.10), when both the joint axes intersect at a point. Skew (fig 4.11), when both the axes of rotation are skew to each other, i.e., the joint axes are neither parallel not intersecting. A special sub-assembly is made for the 6R Spherical Wrist Manipulator. This is known as the wrist sub-assembly. This consists of three joints in which all three joint axes intersect at a point known as the intersecting point. This sub-assembly is used only while generating or creating the robotic arm with the configuration of 6R Spherical Wrist Architecture. The modular components namely, the motor, adapter, motor clamp and links are used to make these sub-assemblies. An electromagnetic gripper [13] is used in this project for simple picking and placing applications. This electromagnetic gripper is currently being controlled with the help of a physical switch which we have to manually operate. The future possibilities of this are to use a Bluetooth switch which will reduce the wiring and improve the utilization of

space.

RAMG (Robot Arm Model Generator) is an algorithm developed by us which is used to generate a virtual model of a robotic arm. This is achieved with the help of an algorithm which takes DH (Denavit – Hartenberg) parameters as input variables. Based on the information received from the DH parameters, the algorithm classifies the model into the appropriate sub-assemblies, and generates the 3D model. The classification done is based on the relative orientation and position of the consecutive joint axes. These classifications are parallel, skew and intersecting.

The process flow of the RAMG is shown below (fig 4.1). In future there is a possible inclusion of adding an Autodesk Inventor add-in, in which the DH parameters which are given as input helps in making the virtual model. This can be predominantly used for education purposes. As this will give students the insight to identify the configuration from the DH parameters given. This drastically increases the kinematical knowledge of the students.

The image of all the components used and the sub-assemblies are shown below.

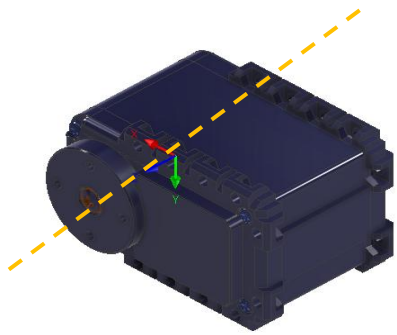


Fig 4.1: Dynamixel AX-12 A motor

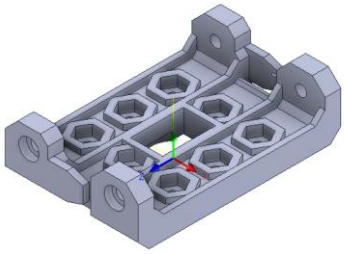


Fig 4.2: Motor's clamp

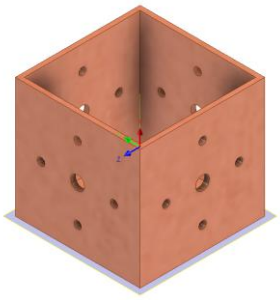


Fig 4.3: Adapter

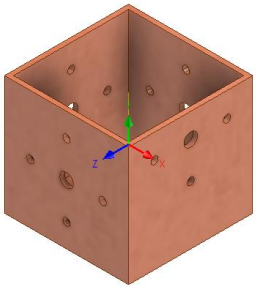


Fig 4.4: Offset Adapter

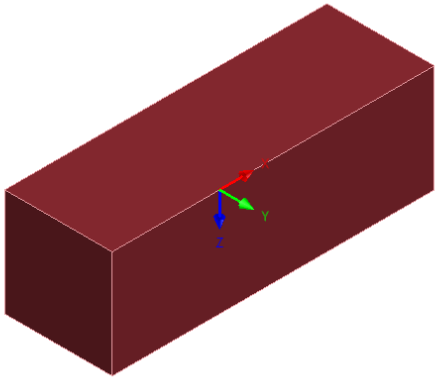


Fig 4.5: Straight Link

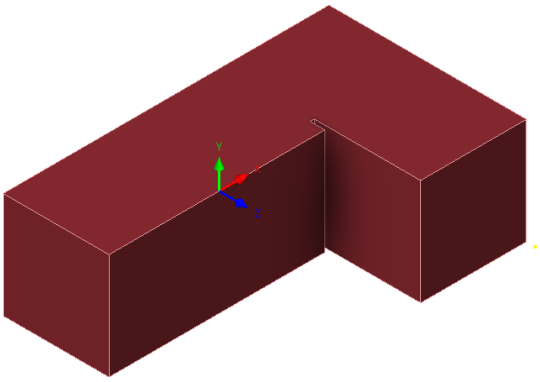


Fig 4.6: Wrist Link

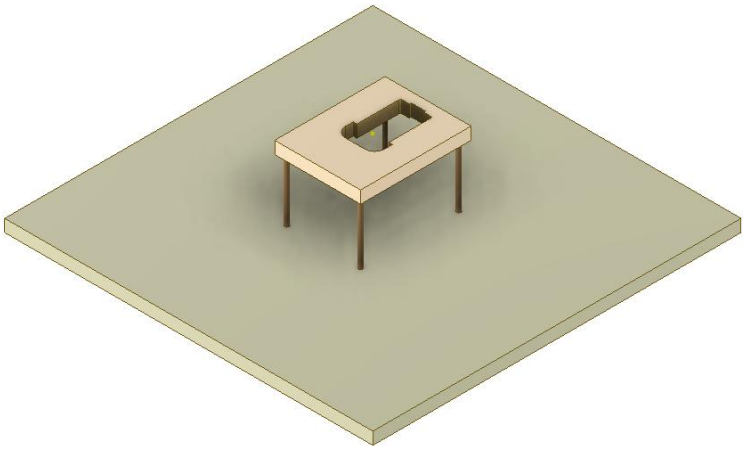


Fig 4.7: Universal Base

Fig 4.8: Electromagnetic Gripper

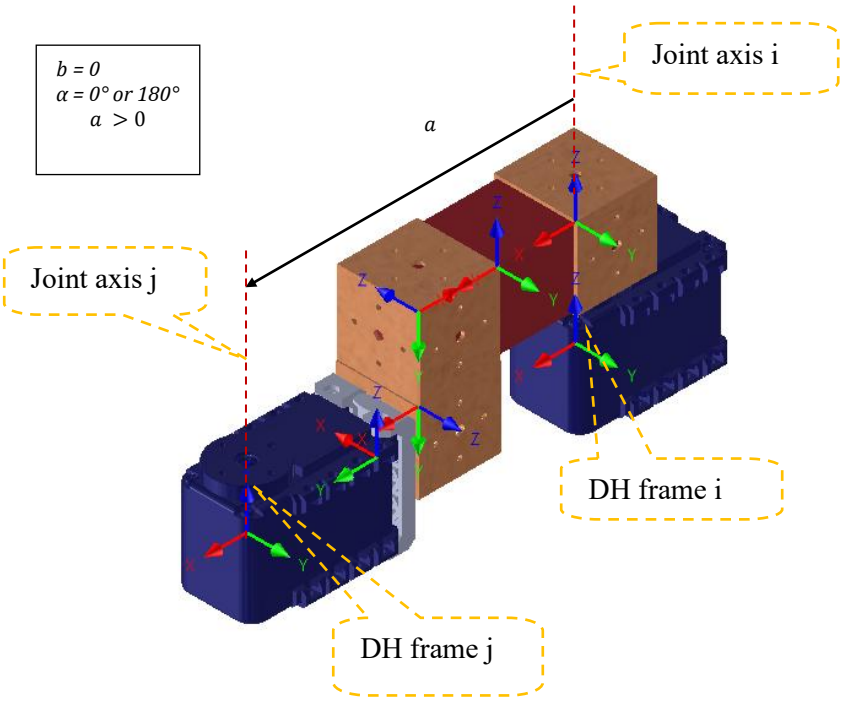


Fig 4.9: Virtual Model of Parallel Sub-Assembly Manipulator

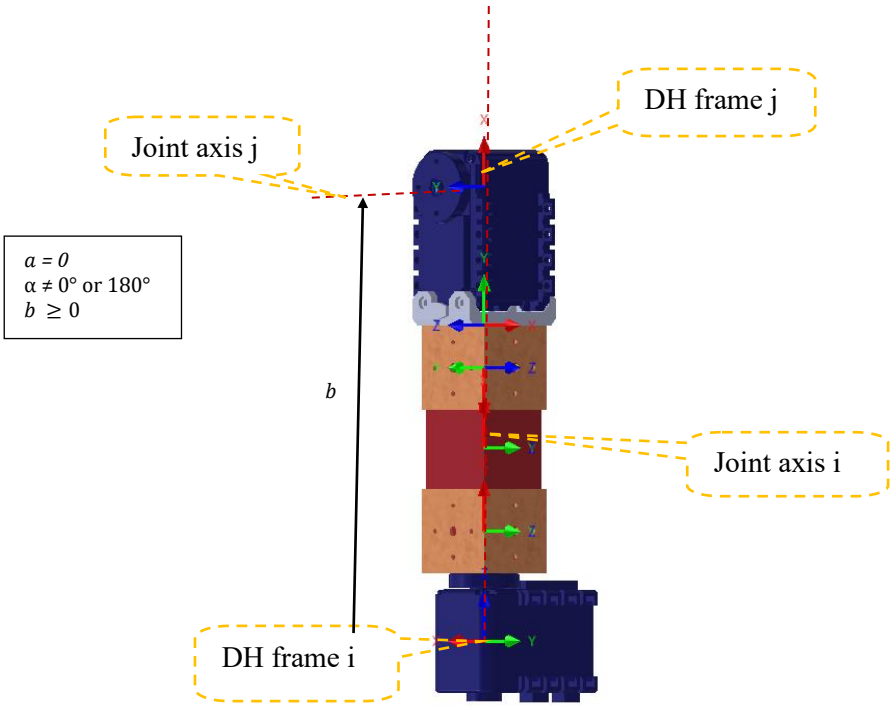


Fig 4.10: Virtual Model of Intersecting type Sub-Assembly.

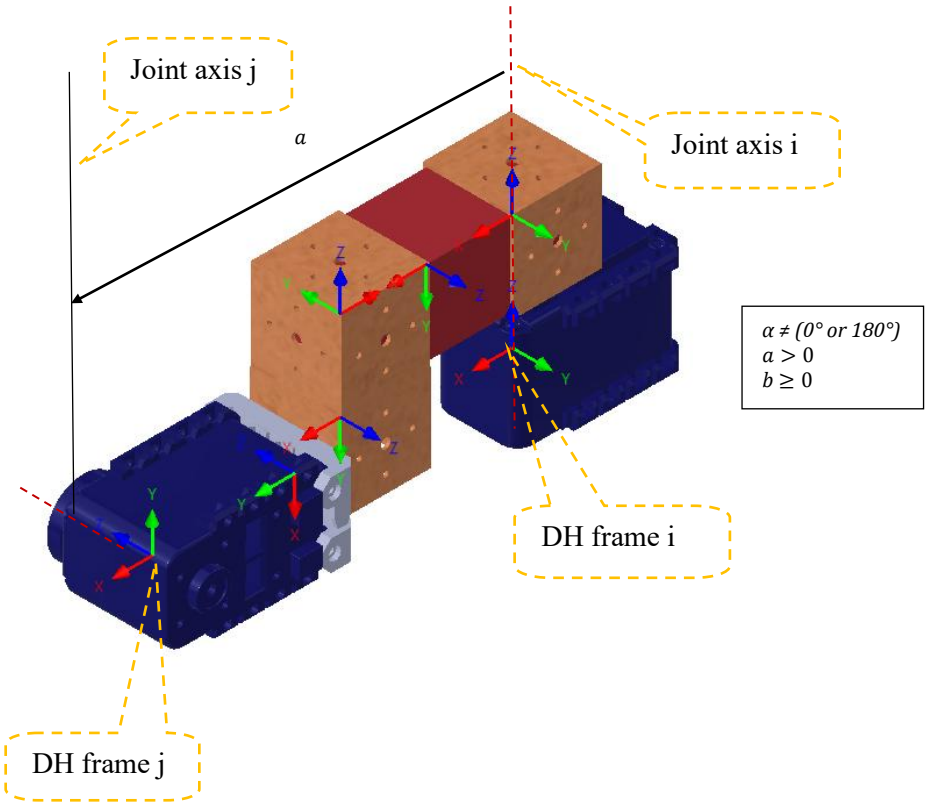


Fig 4.11: Virtual Model of Skew type Sub-Assembly

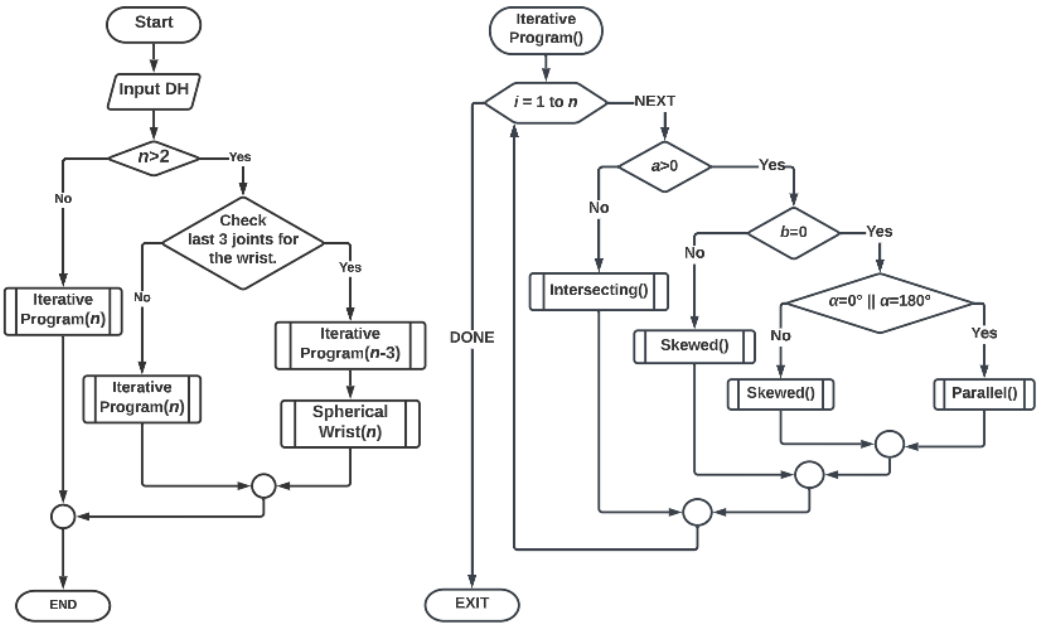


Fig 4.12 : Process flow of RAMG algorithm to selcet the appropriate sub-assembly.

4.1 Virtual Model

In this section, we will be showing the virtual model which is made from the above-mentioned modular components along with the CAD model of the different configurations possible with the modular components. The CAD model of these configuration were designed with the help of the above-mentioned sub-assemblies. These CAD models can be later be converted into actual physical prototypes which are functional with the help of the modular components. The models shown below are 2R planar (fig 4.12), 3R planar (fig 4.13), 3R spatial (fig 4.14), 6R Spherical Wrist (fig 4.15), 6R Offset Wrist (fig 4.16) and dual arm (fig 4.17).

Some of these CAD models are shown below

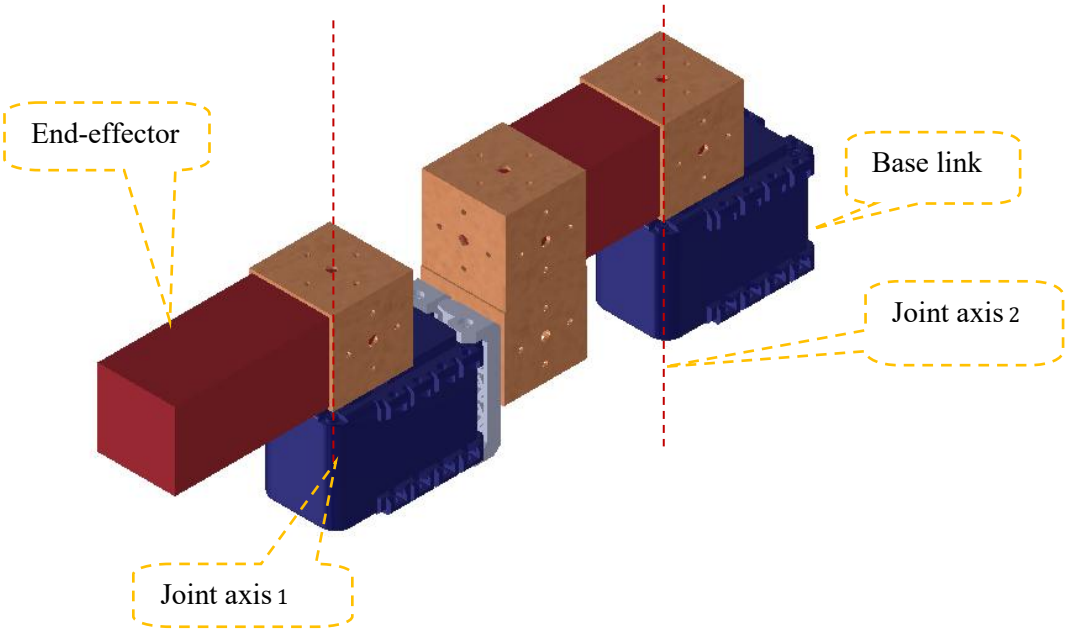


Fig 4.13: Virtual model of 2R Planar

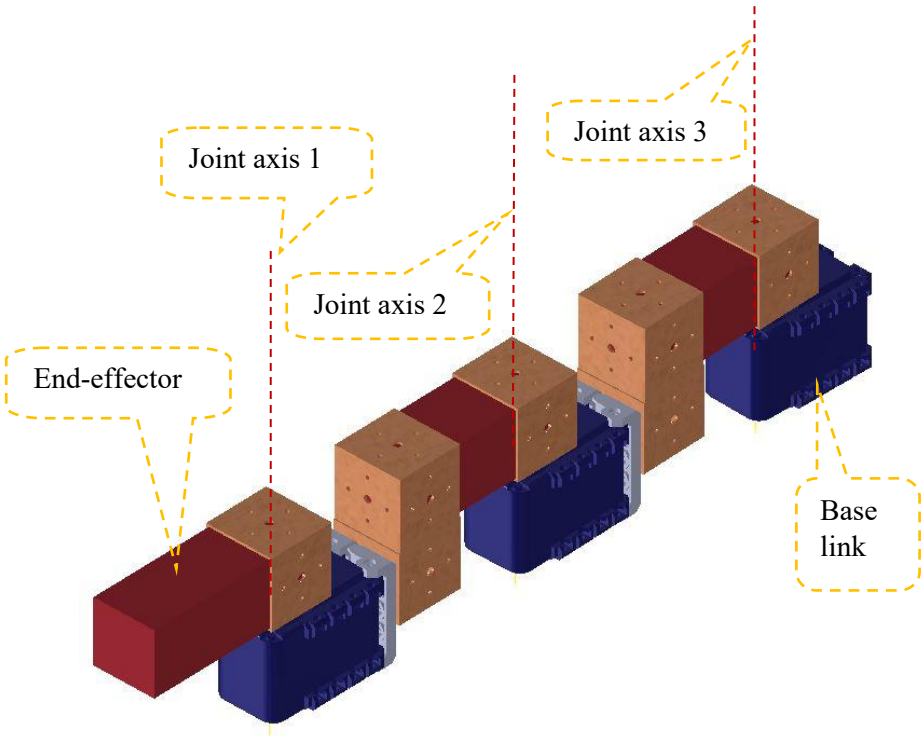


Fig 4.14: Virtual model of 3R Planar Manipulator.

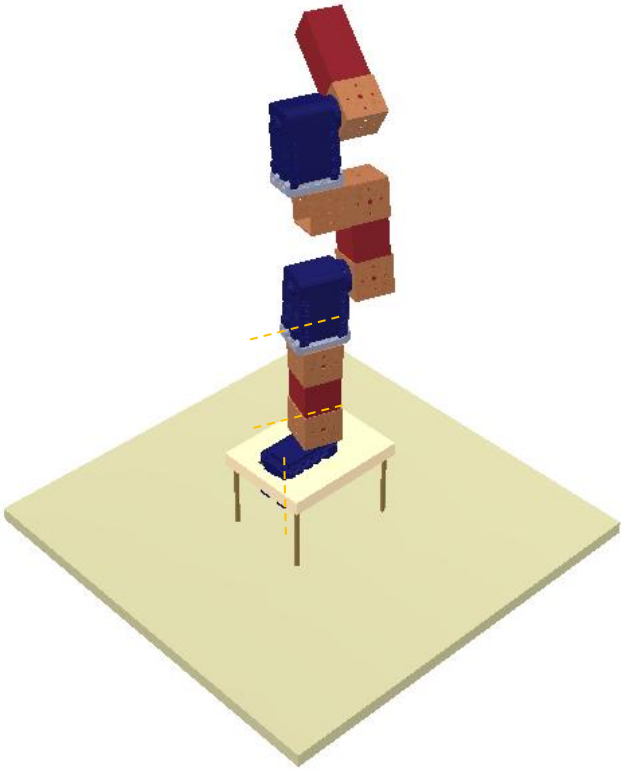


Fig 4.15: 3R Spatial

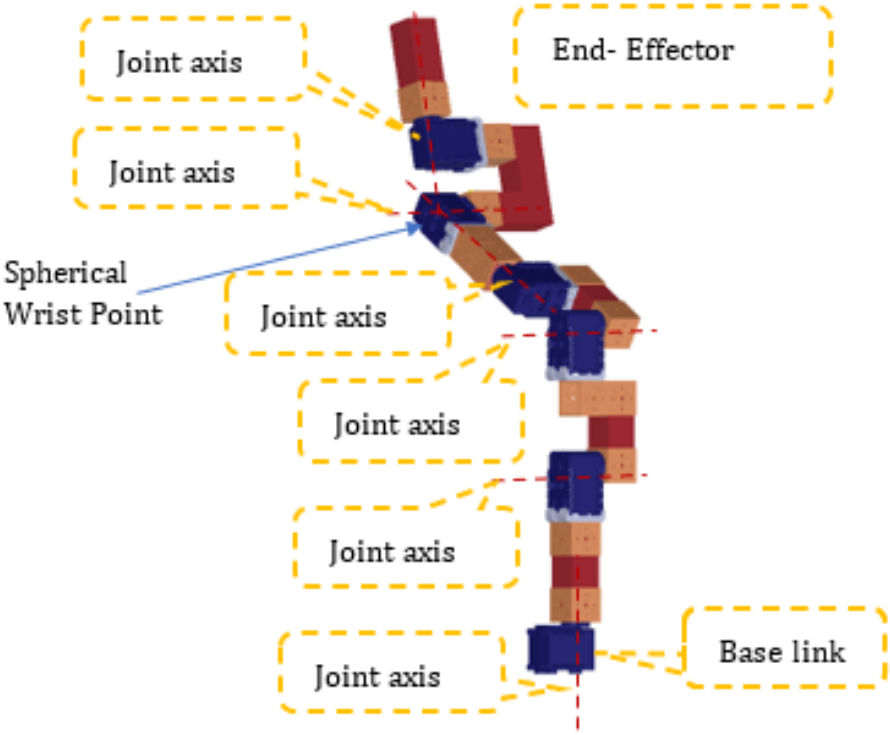


Fig 4.16: Spherical wrist manipulator

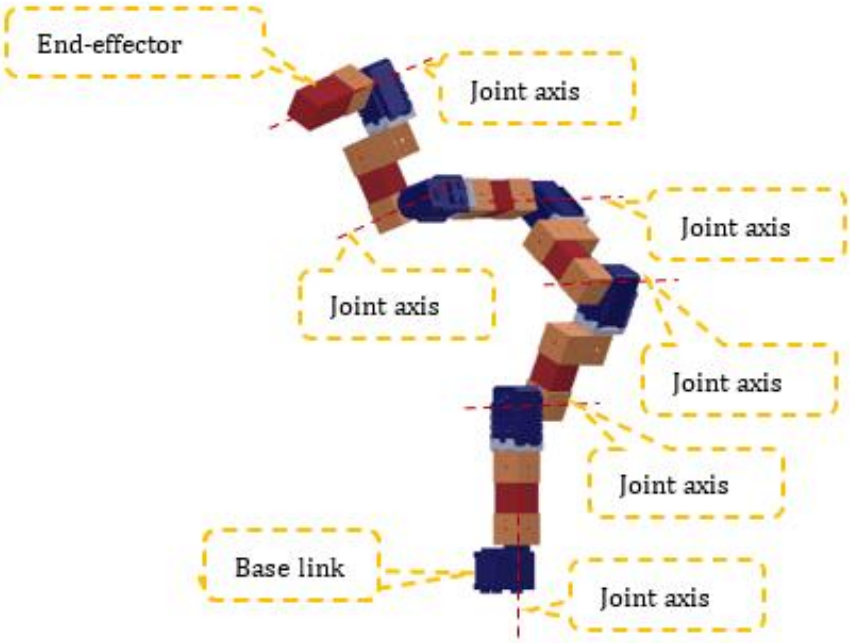


Fig 4.17: Offset wrist manipulator

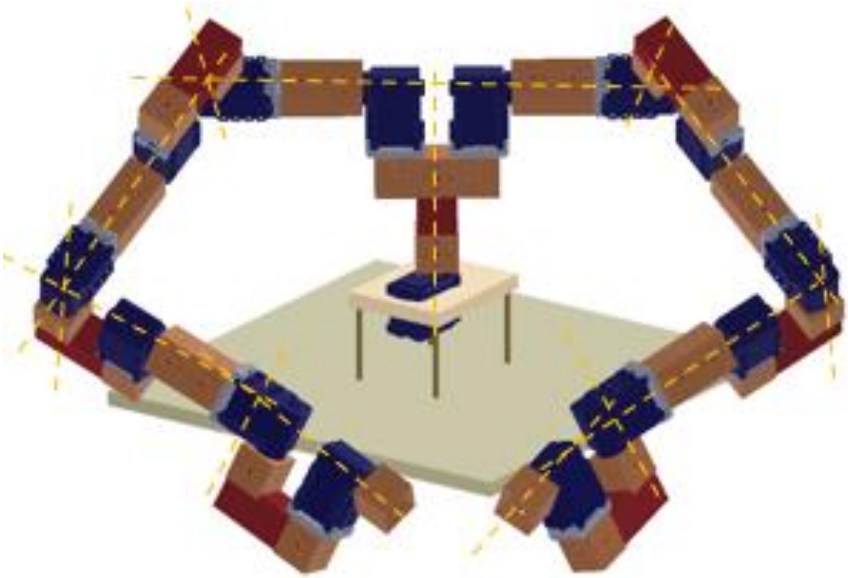


Fig 4.18: Dual Arm Robot (inspired by Yaskawa)

4.2 Physical Prototype

For the students to have a hands-on experience, of a robotic arm, the presence of a physical model is necessary. Hence with the help of the modular components which was mentioned earlier, we can physically assemble a working prototype of the different CAD models which was shown above.

For making the model, the same motor along with the clamps provided with it are used. Screws are used to tighten the different parts together. The links are square rods are made from ABS plastic and PP plastic. These links are used to connect the joint of the robotic arm. The adapters which provide the robotic arm with modularity is made from PP plastic and sheet metal. The base which supports the entire robotic arm is made of plywood with CNC cut slots to mount the motor.

The picture of the physical prototype for multiple configurations is shown below.

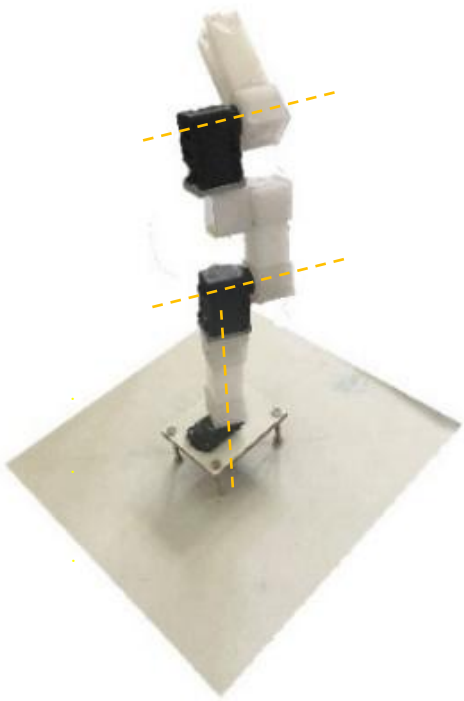


Fig 4.19: 3R Spatial

Fig 4.20: 2R Planar

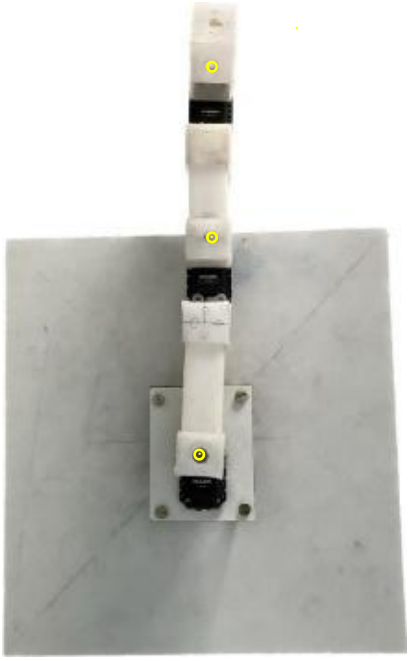


Fig 4.21: 3R Planar

`

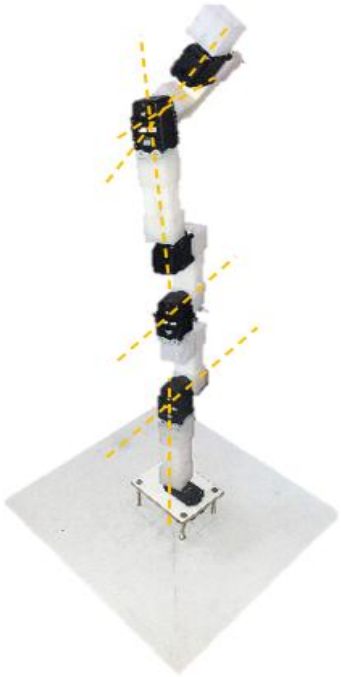


Fig 4.22: Spherical Wrist

Fig 4.23: Offset Wrist

5. Motion Planning and Programming

Motion planning is a very important concept to control the movement of a robot. Motion planning cuts the entire movement of the robot to smaller paths, this helped in understanding the movement of the robot. It also facilitates in the easier programming of the robot. As discussed earlier, depending on the application and user demand's a robotic arm can be programmed with the help of either forward kinematics or inverse kinematics. In this project, we mainly focus on forward kinematics as the robotic arm is mainly used for teaching purposes and for simple pick and place applications like picking a screw and dropping in a cup with the help of an electromagnetic gripper.

To control the manipulator and to move it to the desired position, we use U2D2 controller from Robotis, which is the same company that makes the motors. This controller is a USB communication converter with a PC and power hub board to power up and drive the motors. The parameters of the motor can be changed with the help of this controller and robo plus software, which is an open-source software provided by Robotis. With these, we can change the torque limit, velocity, and multiple other parameters of the motor. This can help us monitor the power consumed by the motor and change the motor parameters depending on the application.

The U2D2 controller was chosen as the motors used in this project are Dynamixel AX-12A from Robotis, which is the same company. Robotis has developed an SDK (Software Development Kit) known as "Dynamixel SDK" to control these motors with the help of a program. The motors are interconnected with the help of daisy chains in which connectors are connected to the from one motor to another, and only the first motor is connected to the controller. This helps with cable management and gives it a clean and tidy look.

This SDK contains a DLL (Dynamic Link Library) file for different programming languages, using which we can develop programs and software to control the motor

thereby articulating the entire robotic arm.

For the sake of convenience and ease of use, in this project, we have developed the programs in C# using Microsoft Visual Studios.

In the C# program, we have created a control class which uses the Dynamixel SDK to send the input values to the correct motors based in the motor's id. This control class acts as the link between the Dynamixel SDK and the forms application. There are functions within the class to simply the process. For creating the CSV file, inbuild commands in C# were used. And the format of the CSV file is changed according to the application which it'll be used for.

With the help of Visual Studios, we have created Windows Forms Application. Two such applications were created to control the manipulator. These are discussed below. A flowchart is shown below which shows the flow of process when the motors are being manipulated.

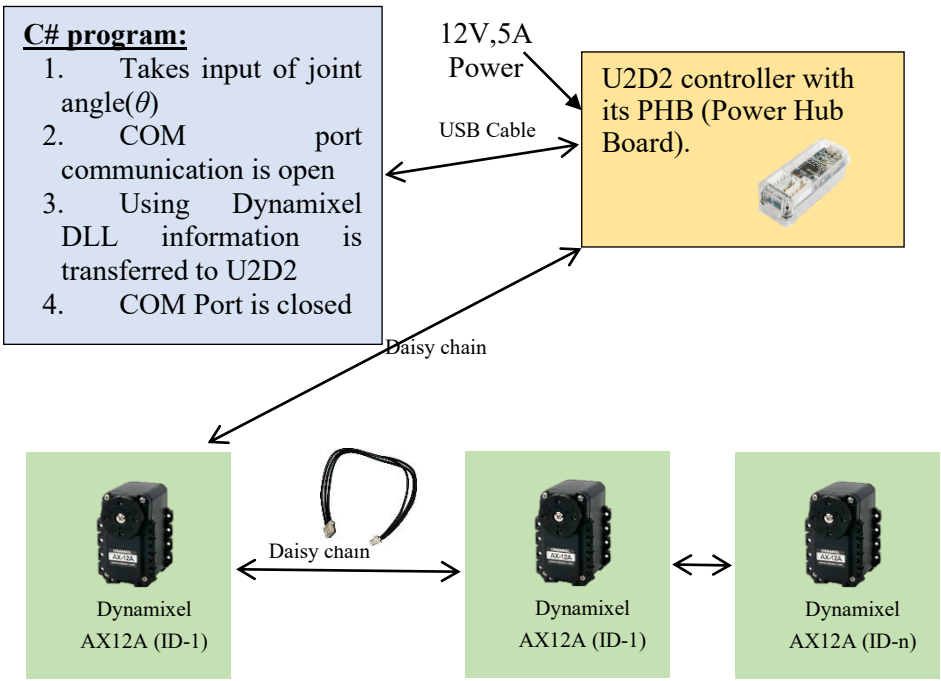


Fig 5.1: Flowchart that represent the flow of process when controlling the motors.

In this project, we have developed programs in C# using Visual Studios. We have

created 2 Windows Forms Applications which are discussed below.

Application 1

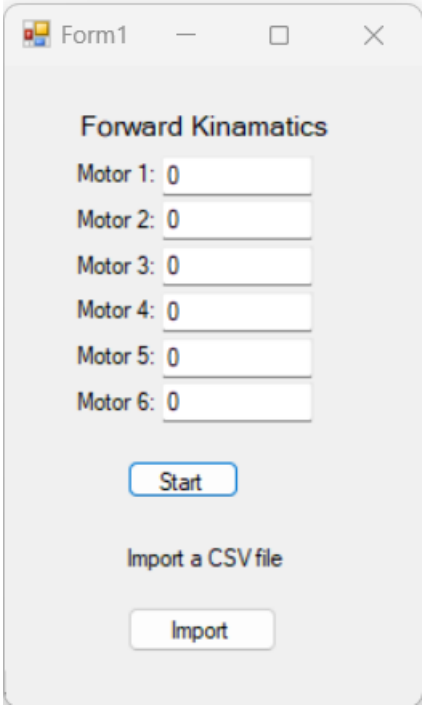


Fig 5.2: Forward Kinematics Forms Application

In this Forms Application, the user is asked to fill in the boxed with the input variables. These are the degrees by which they wish to move the motor. The correct value should be entered adjacent to the correct motors name. after entering the values, when the move button is pressed, the manipulator moves by the set degrees. This forms also has a feature to read a CSV (Comma Separated Values) which contains the joint values along with their step size. When the file is selected and the move button is clicked, the robot is actuated to the position inferred from the data in the CSV file. Depending on the step size, the motion of the manipulator can be seen clearly. This CSV file must be in a specific format and the number columns relates to the number of motors, and the number of rows relate to the total number of steps. Forward kinematics is used in this program to control the robotic arm.

Application 2

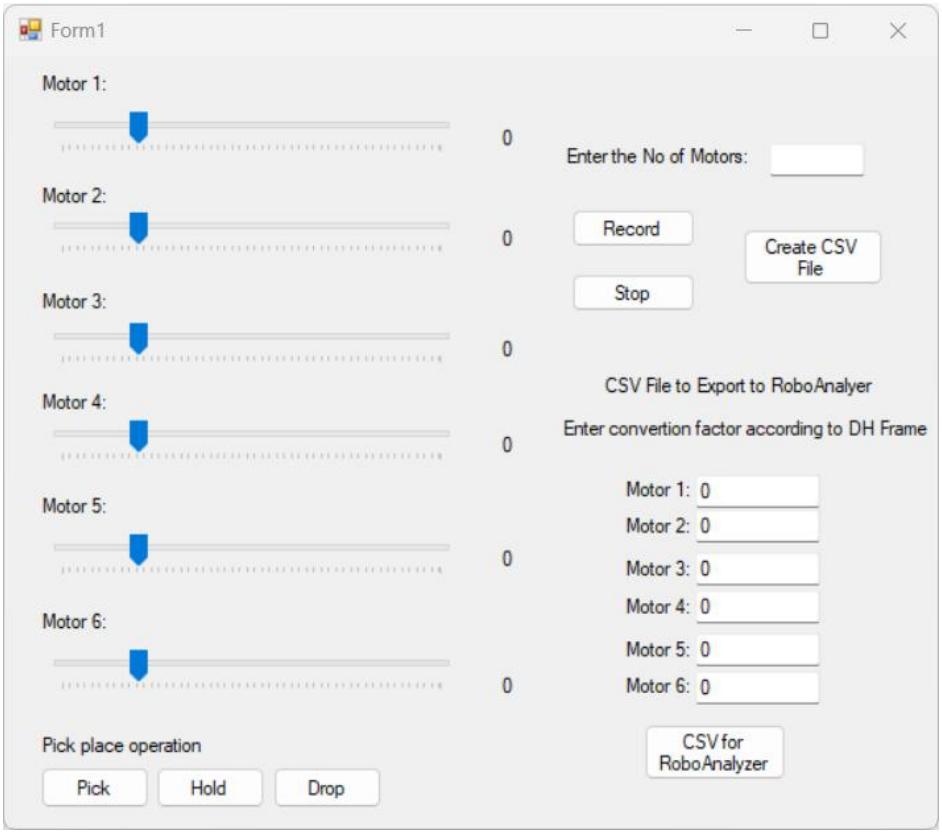


Fig 5.3: Joint Jogging Application

In this form's application, sliders are provided for every motor. These sliders can be used for performing joint jogging i.e., individually moving the motor corresponding to their respective slider. This gives the user absolute manual control over the robotic arm, and can perform any function as desired by the user.

A record button is provided in the forms, when clicked, the program starts recording the movements made by the slider, and the values which corresponds to that is stored in CSV file, in a directory of the user's choice.

There is also an option to export a CSV file which is in the format that is accepted by the robot simulation software RoboAnalyzer.

Using this CSV file, when it is given as input for RoboAnalyzer, a virtual simulation is created for a robotic arm of the same architecture, and the motion of the physical model is mapped to the virtual model which can be seen in the software. This will

give the user an insight of how the simulation software and physical model operate when the same values are given as input.

There are three buttons which are provided in the form which are Pick, Hold and Drop. When Pick is selected, the robotic arm moves to a point which was predefined position in its work envelope and picks up the object from there. When the Hold button is clicked, the manipulator returns to its home position which us upright. Finally, when the Drop button is clicked, the robot moves to another point in its work envelope, where the picked object is dropped into a paper cup. All these three buttons are hard coded; hence their values can be changed only if the program is changed.

6. Conclusion

In the duration of this project, we have had the privilege to publicize our work. This includes two research papers, one science competition and one patent application. This has helped our social outreach and has helped us understand the various steps and precautions taken when doing research. The papers published bares the titles "Modular Serial-Chain Robotic Arm with Plug-and-Play Architecture for Effective Teaching and Learning." In AIR (Advances in Robotics) 2023 at IIT (Indian Institute of Technology), Ropar, India, and "Virtual and Physical Components to Build Modular Robotic Arms for Effective Understanding of Denavit-Hartenberg (DH) Parameters." In IFToMM (International Federation for the Promotion of Mechanism and Machine Science) World Congress 2023, Tokyo, Japan. We have also filed for a provision patent application with the Government of India, under the title "Modular Serial-Chain Robot" bearing the application number 202341022970.

Along with this we were placed second in the International Science Day Competition (Prakalp) held by Amrita School of Engineering, Bangalore in 28th of February 2023.

In summary, we have proposed a new design and method in making a modular robotic arm. The robotic arm is assembled using modular units in different configurations to obtain a specific arm architecture. To operate the robotic arm, The authors have created windows forms applications in Visual studio using C#.

As many types of robotic arms can be created using these modular units, it can be used in laboratories to teach robotics. Concepts related to robotics such as DH parameters, Forward Kinematics, Inverse Kinematics can be studied in detail using the modular robotic arm for multiple arm architectures.

the future scope and work of this project will be to integrate it RoboAnalyzer, for seamless interaction and hands-on experience which will bring cheerful and exuberating teaching and learning experience. This will drastically improve the critical thinking and logical reasoning of the student, which will make them valiant engineers in the future.

Reference

[1]. Denavit, Jacques; Hartenberg, Richard Scheunemann (1955). "A kinematic notation for lower-pair mechanisms based on matrices". Journal of Applied Mechanics. 22 (2): 215–221. doi:10.1115/1.4011045.

[2]. Shruti, Medha, and Rajeevlochana G. Chittawadigi. "Offline Simulation of Motion Planning of a Planar Manipulator in RoboAnalyzer and its Integration with a Physical Prototype." In 2021 IEEE 9th Region 10 Humanitarian Technology Conference (R10-HTC), pp. 01-06. IEEE, 2021.

[3]. http://www.roboanalyzer.com/

[4]. Patwardhan, Amogh, Aditya Prakash, and Rajeevlochana G. Chittawadigi. "Kinematic analysis and development of simulation software for nex dexter robotic manipulator." Procedia computer science 133 (2018): 660-667.

[5]. http://www.nex-robotics.com/products/robotic-arms-and-grippers/dexter-er-1-robotic-arm-with-controller-and-accessories.html

[6]. Chen, I-Ming, and Guilin Yang. "Configuration independent kinematics for modular robots." In Proceedings of IEEE International Conference on Robotics and Automation, vol. 2, pp. 1440-1445. IEEE, 1996.

[7]. Jo, Hyeon Min, Dong Jae Lim, Won Jee Chung, Jong Kap Choi, Dae Young Kim, Yeon Joo Ahn, and Hee Sung Ahn. "Optimal wrist design of wrist-hollow type 6-axis articulated robot using genetic algorithm." In 2018 IEEE International Conference on Mechatronics and Automation (ICMA), pp. 1486-1491. IEEE, 2018.

[8]. Singh, Satwinder, and Ekta Singla. "Modelling and simulation of basic robotic configurations using D–H parameters based adaptive modules." Perspectives in Science 8 (2016): 513-515.

[9]. https://www.kuka.com/9

[10]. https://www.britannica.com/technology/PUMA-robot

[11]. https://www.robotis.us/dynamixel-ax-12a/

[12]. https://emanual.robotis.com/docs/en/parts/interface/u2d2/

[13]. https://robu.in/product/electric-sucker-electromagnet-kk-p20-15-12v/

Appendix.

Appendix A

The control class shown below helps in linking the forms application to the dynamixel DLL, hence acting as a bridge between both.

Control Class

```
using dynamixel_sdk;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Protoype4
{
  public class ControlClass
  {
        // Control table address
      public static int ADDR_MX_TORQUE_ENABLE = 24;
      public static ushort ADDR_MX_GOAL_POSITION = 30;
      public static ushort ADDR_MX_PRESENT_POSITION = 36;

        // Protocol version used by Dynamixel AX-12A
      public const int PROTOCOL_VERSION = 1;

       // Default setting
      public static byte DXL_ID = 1;
      // Dynamixel ID: 1
      public static int BAUDRATE = 1000000;
      public static string DEVICENAME = "COM9";
      // COM port used on PC
      public static ushort DXL_MINIMUM_POSITION_VALUE = 100;
      // Dynamixel will rotate to this value


      public static int port_num = Dynamixel.portHandler(DEVICENAME);
      // To establish Connection to COM port

      public ControlClass(int _ID)
        {
             DXL_ID = Convert.ToByte(_ID);   //Constructor

        }
      public void OpenPort()
      // Opens the port for communication
        {
            Dynamixel.openPort(port_num);
            Dynamixel.setBaudRate(port_num, BAUDRATE);
            Dynamixel.packetHandler();
        }

      public void ClosePort()
      // Close the port for communication
        {
            Dynamixel.closePort(port_num);
```

```
        }
       public void Actuate(int _ID, int pos) // To move motor
        {

            DXL_MINIMUM_POSITION_VALUE = Convert.ToUInt16(pos);
            DXL_ID = Convert.ToByte(_ID);
            UInt16 dxl_goal_position = DXL_MINIMUM_POSITION_VALUE;
             // Goal position

            Dynamixel.write2ByteTxOnly(port_num, PROTOCOL_VERSION,
            DXL_ID, ADDR_MX_GOAL_POSITION, dxl_goal_position);
           // Writes Goal Position

        }
    }
}
```

Appendix B

Forms Application 1 for Forward Kinematics:

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;
using System.Windows.Forms;
using dynamixel_sdk;

namespace Protoype4
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            //we have to initialize ControlClass here
        }

        private void button1_Click(object sender, EventArgs e)
        {    // Assigning user inputs to variables
            Double M1pos = Convert.ToDouble(M1in.Text);
            Double M2pos = Convert.ToDouble(M2in.Text);
            Double M3pos = Convert.ToDouble(M3in.Text);
            Double M4pos = Convert.ToDouble(M4in.Text);
            Double M5pos = Convert.ToDouble(M5in.Text);
            Double M6pos = Convert.ToDouble(M6in.Text);

            int[] Mpos = new int[7];
// Creating 1D Array to pass the angles by converting angles to coded
numbers
```

```
        Mpos[1] = Convert.ToInt32((M1pos * 3.41) + 205.6);
        Mpos[2] = Convert.ToInt32((M2pos * 3.41) + 205.6);
        Mpos[3] = Convert.ToInt32((M3pos * 3.41) + 205.6);
        Mpos[4] = Convert.ToInt32((M4pos * 3.41) + 205.6);
        Mpos[5] = Convert.ToInt32((M5pos * 3.41) + 205.6);
        Mpos[6] = Convert.ToInt32((M6pos * 3.41) + 205.6);
        int id = 0;
         // Assigning Classs
        ControlClass C = new ControlClass(id);
        C.OpenPort();


        for (id = 1; id < 7; id++)
        {
            C.Actuate(id, Mpos[id]);
        }

        C.ClosePort();
    }

    private void ImportButton1_Click(object sender, EventArgs e)
    {
        OpenFileDialog openFileDialog = new OpenFileDialog();
        openFileDialog.Filter = "CSV files (*.csv)|*.csv|All files
         (*.*)|*.*";

        if (openFileDialog.ShowDialog() == DialogResult.OK)
        {
            string filePath = openFileDialog.FileName;
            StreamReader reader = new StreamReader(filePath);
            string fileContents = reader.ReadToEnd();
            reader.Close();

            //2d code below
            string[] lines = fileContents.Split('\n');

            // Determine the number of rows and columns
            int numRows = lines.Length; // Add Subtract 1 for the
            header row if required
            int numCols = lines[0].Split(',').Length;

            // Create a new 2D array to store the imported data
            string[,] data = new string[numRows, numCols];

            // Loop through each row and column, and add the
                values to the array
            for (int i = 0; i < lines.Length; i++)
            {
                string[] values = lines[i].Split(',');
                for (int j = 0; j < values.Length; j++)
                {
                    data[i, j] = values[j];
                }
            }

            // Display the imported data in a message box
            MessageBox.Show("Imported " + numRows + " rows and " +
            numCols + " columns of integers from CSV file.");
```

```
                // Initializing float 2D array
                float[,] floatData = new float[(data.GetLength(0)),
                (data.GetLength(1))];

                // Looping each element of string
                for (int i = 0; i < data.GetLength(0); i++)
                {
                    for (int j = 0; j < data.GetLength(1); j++)
                    {
                        // Convert the string element to a float and
                           store it in the float 2D array
                        float.TryParse(data[i, j], out floatData[i,j]);
                    }
                }
                int id = 1;
                int pos = 0;
                ControlClass C = new ControlClass(id);
                C.OpenPort();
                for (int i = 0; i < floatData.GetLength(0); i++)
                {
                    for (int j = 0; j < floatData.GetLength(1); j++)
                    {
                        pos = Convert.ToInt32((float)floatData[i, j] *
                         3.41 + 205.65);
                        id = j+1;
                        C.Actuate(id, pos);
                        Thread.Sleep(1);    // to slow by 1 ms
                    }
                }
            C.ClosePort();
            }

        }
    }
}
```

Appendix C

Forms Application 2 for joint jogging:

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using dynamixel_sdk;
using static System.Windows.Forms.VisualStyles.VisualStyleElement;

namespace Protoype4
```

```
{

    public partial class Form1 : Form
    {
        int id = 1;
        int Rec = 0;
        int R = 0;
        double[,] RecArr = new double[2000, 6];
        int NoOfMotor = 0;

        public Form1()
        {
            InitializeComponent();
            ControlClass C = new ControlClass(id);
            C.OpenPort();

            //we have to initialize ControlClass here
        }

        private void trackBarM1_Scroll(object sender, EventArgs e)
        {
            ControlClass C = new ControlClass(id);
            C.OpenPort();
            int g1;
            double g2;
            g1 = Convert.ToInt32(trackBarM1.Value);
            g2 = (g1 - 205.6) / 3.41;
            M1tracker.Text = Convert.ToString(Math.Round(g2));
            if (Rec == 1)
            {
                RecArr[R, 0] = g2;
                R++;
            }
            C.Actuate(1, g1);

        }

        private void trackBarM2_Scroll(object sender, EventArgs e)
        {
            ControlClass C = new ControlClass(id);
            C.OpenPort();
            int g1;
            double g2;
            g1 = Convert.ToInt32(trackBarM2.Value);
            g2 = (g1 - 205.6) / 3.41;
            M2tracker.Text = Convert.ToString(Math.Round(g2));
            if (Rec == 1)
            {
                RecArr[R, 1] = g2;
                R++;
            }
            C.Actuate(2, g1);

        }

        private void trackBarM3_Scroll(object sender, EventArgs e)
        {
            ControlClass C = new ControlClass(id);
            C.OpenPort();
```

```
            int g1;
            double g2;
            g1 = Convert.ToInt32(trackBarM3.Value);
            g2 = (g1 - 205.6) / 3.41;
            M3tracker.Text = Convert.ToString(Math.Round(g2));
            if (Rec == 1)
            {
                RecArr[R, 2] = g2;
                R++;
            }
            C.Actuate(3, g1);

        }

        private void trackBarM4_Scroll(object sender, EventArgs e)
        {

            ControlClass C = new ControlClass(id);
            C.OpenPort();
            int g1;
            double g2;
            g1 = Convert.ToInt32(trackBarM4.Value);
            g2 = (g1 - 205.6) / 3.41;
            M4tracker.Text = Convert.ToString(Math.Round(g2));
            if (Rec == 1)
            {
                RecArr[R, 3] = g2;
                R++;
            }
            C.Actuate(4, g1);

        }
        private void trackBarM5_Scroll(object sender, EventArgs e)
        {
            ControlClass C = new ControlClass(id);
            C.OpenPort();
            int g1;
            double g2;
            g1 = Convert.ToInt32(trackBarM5.Value);
            g2 = (g1 - 205.6) / 3.41;
            M5tracker.Text = Convert.ToString(Math.Round(g2));
            if (Rec == 1)
            {
                RecArr[R, 4] = g2;
                R++;
            }
            C.Actuate(5, g1);
        }

        private void trackBarM6_Scroll(object sender, EventArgs e)
        {
            ControlClass C = new ControlClass(id);
            C.OpenPort();
            int g1;
            double g2;
            g1 = Convert.ToInt32(trackBarM6.Value);
            g2 = (g1 - 205.6) / 3.41;
            M6tracker.Text = Convert.ToString(Math.Round(g2));
            if (Rec == 1)
```

```
        {
            RecArr[R, 5] = g2;
            R++;
        }
        C.Actuate(6, g1);
    }

    private void PickB_Click(object sender, EventArgs e)
    {
        ControlClass C = new ControlClass(id);
        C.OpenPort();
        C.Actuate(1, 402);
        C.Actuate(2, 567);
        C.Actuate(3, 448);
        C.ClosePort();
    }

    private void HoldB_Click(object sender, EventArgs e)
    {
        ControlClass C = new ControlClass(id);
        C.OpenPort();
        C.Actuate(1, 205);
        C.Actuate(2, 205);
        C.Actuate(3, 205);
        C.ClosePort();
    }

    private void DropB_Click(object sender, EventArgs e)
    {
        ControlClass C = new ControlClass(id);
        C.OpenPort();
        C.Actuate(1, 912);
        C.Actuate(2, 519);
        C.Actuate(3, 205);
        C.ClosePort();
    }

    private void Record_Click(object sender, EventArgs e)
    {
        Rec = 1;
    }

    private void RecStop_Click(object sender, EventArgs e)
    {
        Rec = 0;
    }


    private void Create_CSV_Click(object sender, EventArgs e)
    {
        NoOfMotor = Convert.ToInt32(NoOfMotors.Text);


        SaveFileDialog saveFileDialog = new SaveFileDialog();
        saveFileDialog.Filter = "CSV files (*.csv)|*.csv";
        if (saveFileDialog.ShowDialog() == DialogResult.OK)
        {
            using (StreamWriter writer = new
            StreamWriter(saveFileDialog.FileName))
```

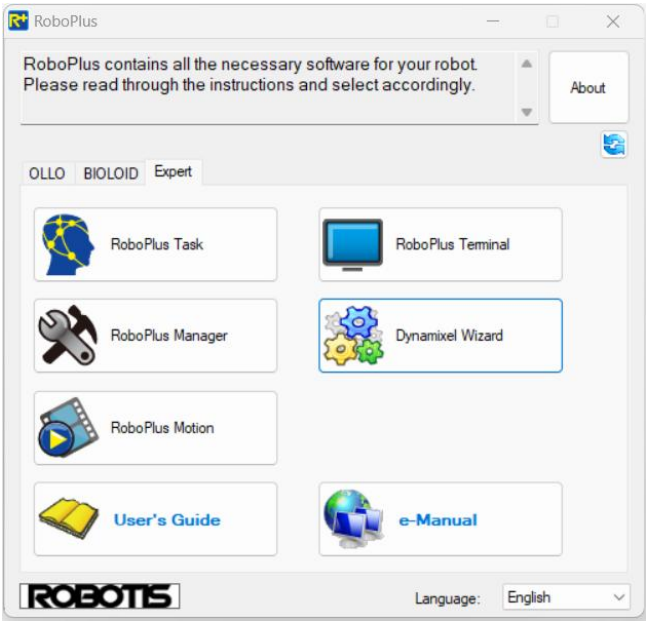
```
            {
                // Writing comma-separated CSV file
                for (int i = 0; i < R; i++)
                {
                    for (int j = 0; j < NoOfMotor; j++)
                    {
                        writer.Write(RecArr[i, j]);
                        if (j < RecArr.GetLength(1) - 1)
                        {
                            writer.Write(",");
                        }
                    }
                    writer.WriteLine();
                }

                // Displays Status
                MessageBox.Show("CSV file is created
                successfully!", "Success", MessageBoxButtons.OK,
                MessageBoxIcon.Information);
            }
        }
    }

    private void RoboAnalyzerCSV_Click(object sender, EventArgs e)
    {
        double temp = 0.5;
        int Converter = 0;  // to use in for loop for conversion;
        int NoOfCol = ((Convert.ToInt32(NoOfMotors.Text) * 3) + 1);
        double[,] ExportArr = new double[R, NoOfCol];
        for (int i = 0; i < R; i++) // to create Time column
        {
            ExportArr[i, 0] = temp;
            temp = temp + 0.5;
        }

        for (int i = 0; i < R; i++)
        {
            Converter = 0;
            for (int j = 1; j < NoOfCol; j = j + 3)
            {

                ExportArr[i, j] = RecArr[i, Converter];
                Converter++;
            }


        }
        //Array is created but factors have to be added below

        double[] FArr = new double[6];
        FArr[0] = Convert.ToDouble(F1.Text);
        FArr[1] = Convert.ToDouble(F2.Text);
        FArr[2] = Convert.ToDouble(F3.Text);
        FArr[3] = Convert.ToDouble(F4.Text);
        FArr[4] = Convert.ToDouble(F5.Text);
        FArr[5] = Convert.ToDouble(F6.Text);

        int FAdder;
        for (int i = 0; i < R; i++)
```

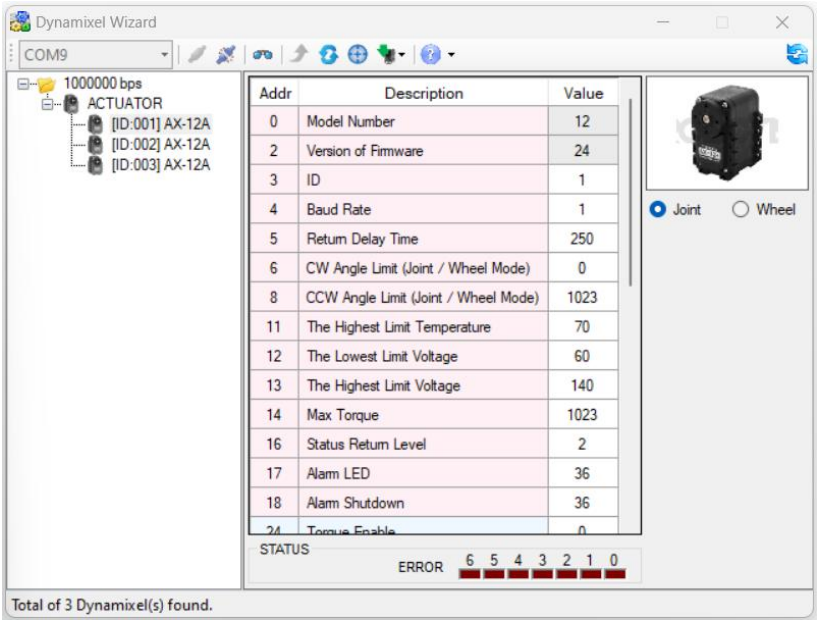
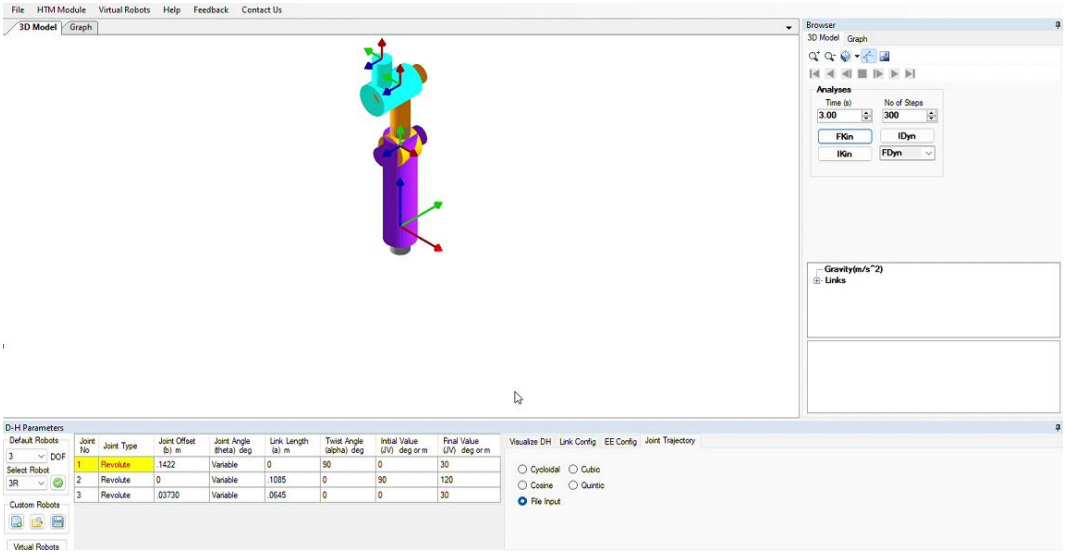
```
            {
                FAdder = 0;
                for (int j = 1; j < NoOfCol; j = j + 3)
                {
                    ExportArr[i, j] = ExportArr[i, j] + FArr[FAdder];
                    FAdder++;
                }
            }
            // the 2D array is ready to be exported

            SaveFileDialog saveFileDialog = new SaveFileDialog();
            saveFileDialog.Filter = "CSV files (*.csv)|*.csv";
            if (saveFileDialog.ShowDialog() == DialogResult.OK)
            {
                using (StreamWriter writer = new
                StreamWriter(saveFileDialog.FileName))

                {
                    // Writing CSV file

                    for (int i = 0; i < R; i++)
                    {
                        for (int j = 0; j < NoOfCol; j++)
                        {
                            writer.Write(ExportArr[i, j]);
                            if (j <ExportArr.GetLength(1) - 1)
                            {
                                writer.Write(",");
                            }
                        }
                        writer.WriteLine();
                    }

                    // Status Indicator
                   MessageBox.Show("CSV file is created
                    successfully!", "Success", MessageBoxButtons.OK,
                    MessageBoxIcon.Information);
                }
            }
        }
    }
}
```

Appendix D

RoboPlus: Software provided by Robotis, wherein the motors ID, Torque, position, and other details related to the motor are shown and can be varied depending on the users choice. This is done under the Dynamixel Wizard tab which searches for the motor via COM port set by the user.

This is shown below:





RoboAnalyzer: A robot simulation software specially used to teach Robotics. We have learnt robot kinematics using this and have used this software to create and simulate a virtual model of the physical robot. It is done by inputting the DH parameters and uploading a CSV file that is generated by the created above application. The joint jogging that is recorded in the physical model can be replicated in the virtual
model.